

DSA STUDY BLUEPRINT SERIES

42. Combination Sum Backtracking

Unique Combinations summing up to Target

Section 06 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Backtracking Choice:** Pick elements multiple times or skip them.
- Sort elements to prevent duplicate combinations.

Memory & Execution Blueprint

Target: 8 • Candidates: [2, 3, 5] $\Rightarrow$ Path 2+2+2+2=8
(Found!)

C++ Implementation Reference

Standard code layout and syntax

```
void getCombinations(vector& cand, int i, int target, vector& curr, vector>& ans) {  
    if (target == 0) { ans.push_back(curr); return; }  
    if (i == cand.size() || target < 0) return;  
    curr.push_back(cand[i]);  
    getCombinations(cand, i, target - cand[i], curr, ans); // Pick again  
    curr.pop_back();  
    getCombinations(cand, i + 1, target, curr, ans); // Skip  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(2^{T/Min})$
Space Complexity	$O(T/Min)$ stack

Key Practices & Gotchas

- **Important:** Sorting candidates allows easy pruning when current element > target.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace recursive decision tree transitions generating subsets for [1, 2] :

Decision Level	Element choice	Recursive action	Subsets array state
i = 0	Include 1	Recurse level 1	[[1]]
i = 1	Exclude 2	Base case hit	Add subset [1] to results

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **State Reversals:** Essential backtrack actions pop choice selections to restore node states before adjacent searches.
- **Pruning:** Skip duplicate recursive paths by checking elements duplicates before branching.
- **Related LeetCode Problems:** #78 Subsets, #90 Subsets II, #46 Permutations.